

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**  
Department of Computer Science and Engineering

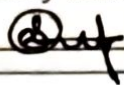
**ASSESSMENT TOOL 3 - SHORT ANSWER TEST**

|                  |                                                                                                              |               |                                       |
|------------------|--------------------------------------------------------------------------------------------------------------|---------------|---------------------------------------|
| Course Code:     | CSA0305                                                                                                      | Course Title: | Data Structures                       |
| CO Assessed:     | CO5 - Manipulation (BL3, BL4)                                                                                | Assessment:   | Assessment Tool 3 - Short Answer Test |
| Weightage:       | 30% of CIA                                                                                                   | Total Marks:  | 100                                   |
| CO2 Statement:   | Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem. |               |                                       |
| Student Name:    | K. Sushesika                                                                                                 | Reg. No.:     | 192511070                             |
| Section / Batch: | BE CSE                                                                                                       | Date:         | 04/09/2026                            |

**Code of Conduct**

I K. Sushesika (192511070) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

**Instructions**

- This test contains 10 short answer test questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

| Section / Topic         | Focus                         | Q Nos | Marks            |
|-------------------------|-------------------------------|-------|------------------|
| Minimum Spanning Tree   | Kruskal's Algorithm           | 1     | 20               |
| Graph Sorting           | Topological Sort              | 2     | 20               |
| Shortest Path Algorithm | MST & Dijkstra's Algorithm    | 3     | 20               |
| Shortest Path Algorithm | Dijkstra's Algorithm          | 4     | 20               |
| Minimum Spanning Tree   | Prim's or Kruskal's Algorithm | 5     | 20               |
| <b>GRAND TOTAL:</b>     |                               |       | <b>100 Marks</b> |

## Annexure A - Short Answer Test

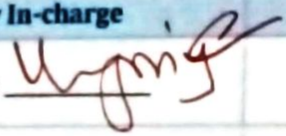
1. Perform the following operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operations: 1) Invert the elements in the stack, 2) Pop (), 3) Pop (), 3) Pop Construct AVL tree for the following elements. Mention the
2. Given a collection of elements 10,20,30,40,50,60,70,80. The Key element to be search 70. Compare how searching is performed in Linear Search and Binary Search. Identify the efficient search method. Justify.
3. What is the postfix and prefix forms of the following expression?  $A+B*(C-D)/(P-R)$
4. Construct AVL tree for the following elements. Mention the Rotations in each step of insertion. 1,2,3,4,5,6,7,8,9,10
5. Construct a heap by inserting 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45 into a binary heap.
6. Construct a B-Tree of Order 3 by inserting numbers 34,26,45,12,87,56,78,22,23,24,65,85,95.
7. Perform the merge sort for the following inputs. Also, show the result for each step of iteration. 40,20,70,14,60,61,97,30.
8. How the following data set can be sorted by quick sort. Show each step-in sorting. 78,23,31,88,43,55,67,55.
9. Perform Open Hashing & Linear Probing for the given set of 0,1,81, 4,64,25,16,36, 9 and 49.
10. Consider the following graph & construct BFS.



Graph with six nodes

### Rubrics for Calculation

| Criteria                     | Weightage | Level 4 - Exemplary                                                    | Level 3 - Proficient               | Level 2 - Developing           | Level 1 - Beginning           |
|------------------------------|-----------|------------------------------------------------------------------------|------------------------------------|--------------------------------|-------------------------------|
| Conceptual Understanding     | 30        | Demonstrates complete and accurate understanding of concepts           | Good understanding with minor gaps | Basic understanding            | Limited understanding         |
| Accuracy of Answer           | 25        | Answers are completely correct and relevant                            | Mostly correct with minor errors   | Partially correct              | Mostly incorrect              |
| Problem-Solving & Reasoning  | 20        | Provides logical reasoning and appropriate steps                       | Good reasoning with minor gaps     | Basic reasoning                | Lacks logical reasoning       |
| Clarity & Relevance          | 15        | Answers are precise, clear, concise, and directly address the question | Mostly clear and relevant          | Somewhat unclear or incomplete | Irrelevant or unclear answers |
| Time Management & Completion | 10        | Completes all questions accurately within the allotted time            | Completes most questions on time   | Requires additional time       | Unable to complete the test   |

| Faculty In-charge                                                                              | Course Coordinator | Program Director |
|------------------------------------------------------------------------------------------------|--------------------|------------------|
| Signature:  | Signature: _____   | Signature: _____ |
| Date: _____                                                                                    | Date: _____        | Date: _____      |



# CO5 - Assessment

## Tool - 3

Name : K. Sushesika

Reg No : 192511070

Code : CSA0305

Course : Data Structures.

### 1) Stack Operations.

Stack size = 5

Initial stack from position 0 to 4 :

| Position | 0  | 1  | 2  | 3  | 4  |
|----------|----|----|----|----|----|
| Element  | 22 | 55 | 33 | 66 | 88 |

Assuming position 4 is TOP

Step 1: Invert the Stack

After inversion:

| Position | 0  | 1  | 2  | 3  | 4  |
|----------|----|----|----|----|----|
| Element  | 88 | 66 | 33 | 55 | 22 |

TOP = 22.

Step 2: POP()

\* Popped element = 22

\* Stack : 88, 66, 33, 55

\* TOP = 55

Step 3: POP()

\* Popped element = 55

\* Stack : 88, 66, 33

TOP = 55

Step 3: Pop()

\* Popped element = 55

\* Stack: 88, 66, 33

TOP = 33

Step 4: Pop()

\* Popped element = 33

Stack: 88, 66

TOP = 66

Final Stack

TOP



66

88

Popped elements in order: 22 → 55 → 33

Construct AVL Tree

Elements: 22, 55, 33, 66, 88

Step 1: Insert 22

(22)

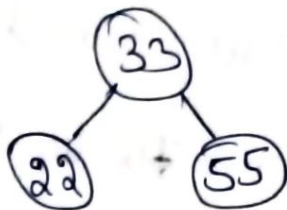
Step 2: Insert 55

(22)

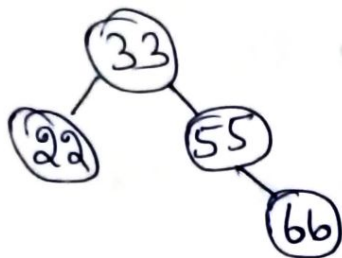
(55)

Step 3: Insert 33

This creates an RL (Right-Left) imbalance at 22.  
After RL rotation:

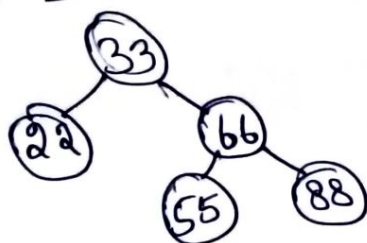


Step 4: Insert 66



Step 5: Insert 88  
This causes an RR imbalance at 55. Perform a left rotation

Final AVL Tree



2) Gm: 10, 20, 30, 40, 50, 60, 70, 80  
Key to search: 70

Linear Search

↳ It checks each element one by one from the beginning.

10 → 20 → 30 → 40 → 50 → 60 → 70 ✓

\*Comparisons required = 7

\*Works on both sorted and unsorted data.

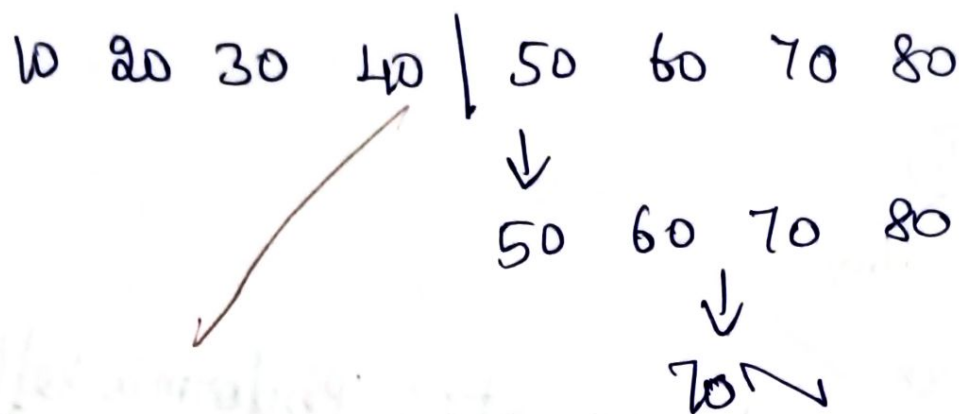


\* Time Complexity =  $O(n)$ .

## Binary Search

↳ It requires the elements to be sorted.

1. Middle element = 40  $\rightarrow 70 > 40$ , search right half
2. Middle of 50, 60, 70, 80 = 60  $\rightarrow 70 > 60$ , search right
3. Middle = 70  $\rightarrow$  Found  $\searrow$



Comparisons required = 3

Time Complexity =  $O(\log n)$ .

3) Grn:

$$A + B \times (C - D) / (P - R)$$

Prefix Form

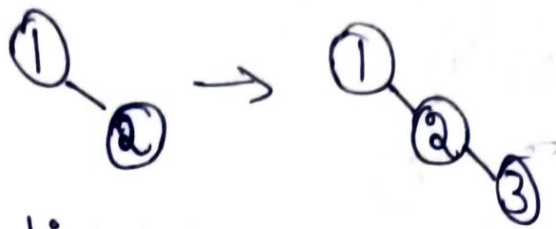
$$+ A / * B - C D - P R$$

Postfix Form

$$A B C D - * P R - / +$$

# AVL Tree

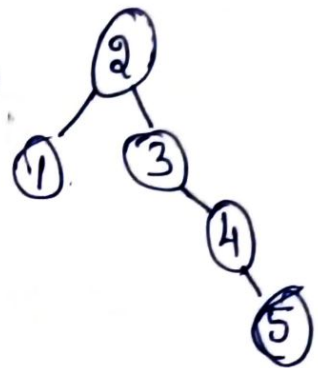
Elements: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10



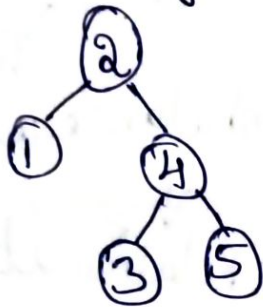
After inserting 2:



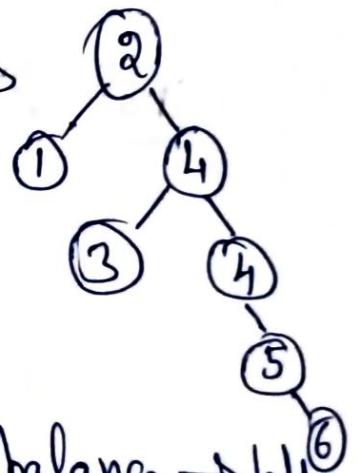
RR imbalance  $\rightarrow$  left rotation at 1  $\rightarrow$



After inserting 5:

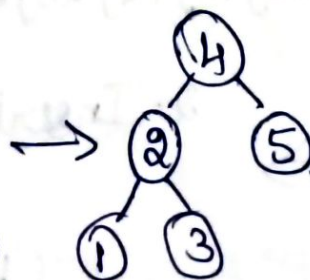
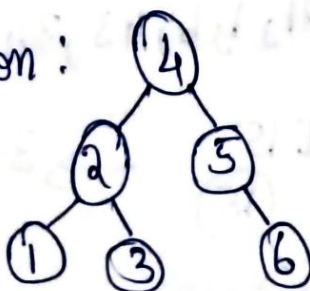


RR imbalance  $\rightarrow$  left rotation at 3  $\rightarrow$



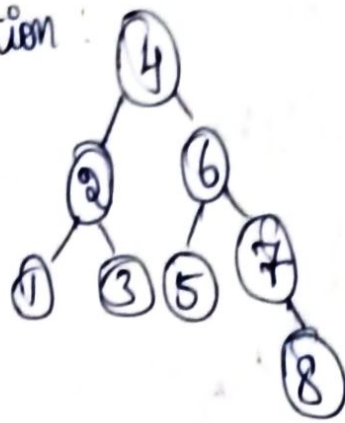
RR imbalance  $\rightarrow$  left rotation at 2

After rotation:



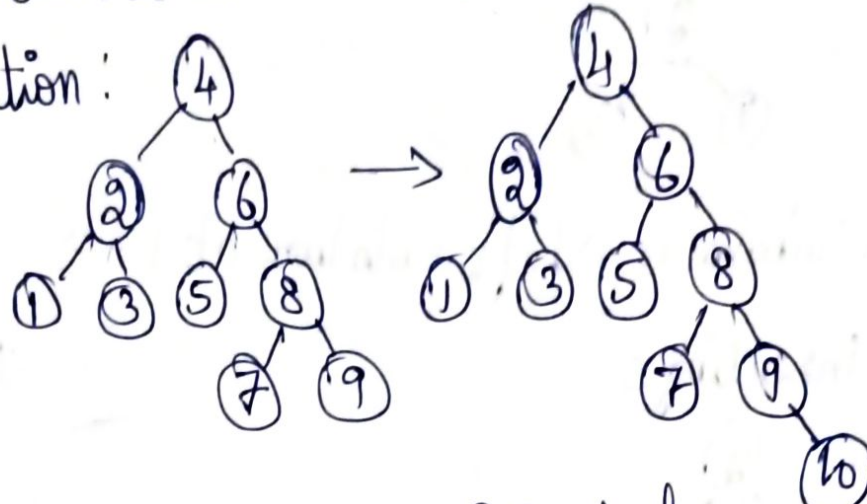
RR imbalance at 5  $\downarrow$  Left Rotation at 5

After rotation:



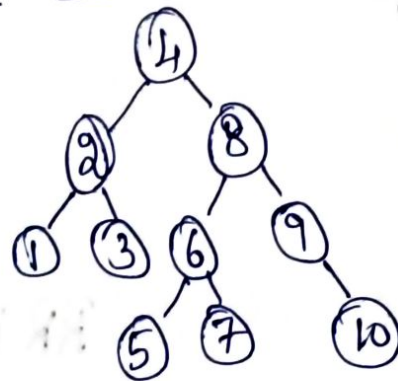
RR imbalance at 7 → Left Rotation at 7.

After rotation:



RR imbalance at 6  
↓  
Left Rotation at 6.

Final AVL Tree.

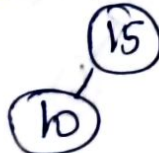


5) Elements: 10, 15, 12, 23, 21, 1, 6, 43, 34, 56, 78, 23, 45.

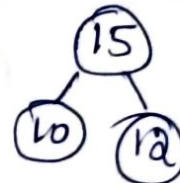
1. Insert 10



2. Insert 15

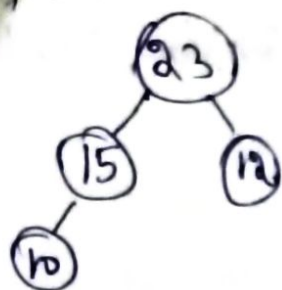


→ 3. Insert 12

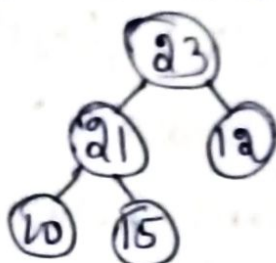




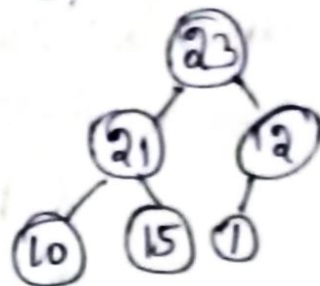
1. Insert 23



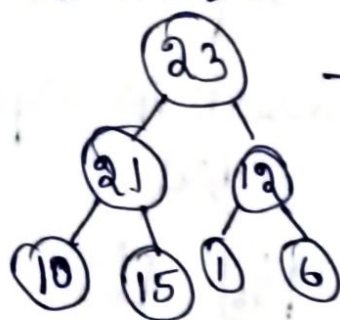
5. Insert 21



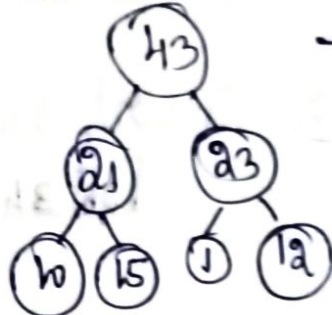
6. Insert 1



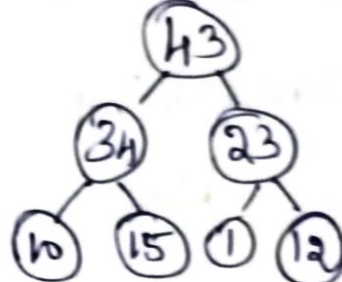
7. Insert 6



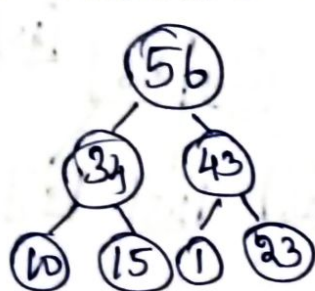
8. Insert 43



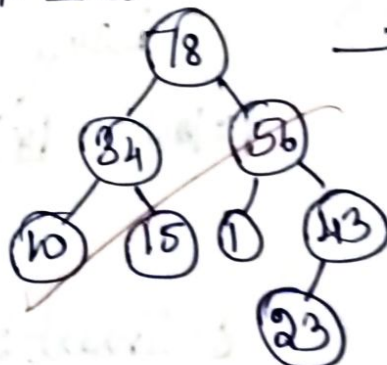
9. Insert 34



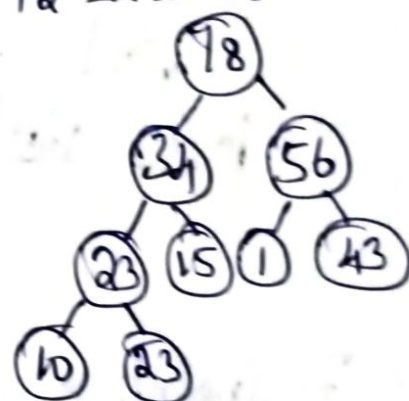
10. Insert 56



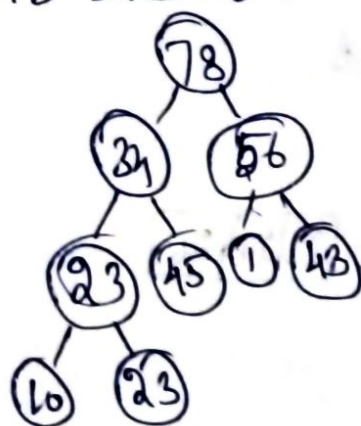
11. Insert 78



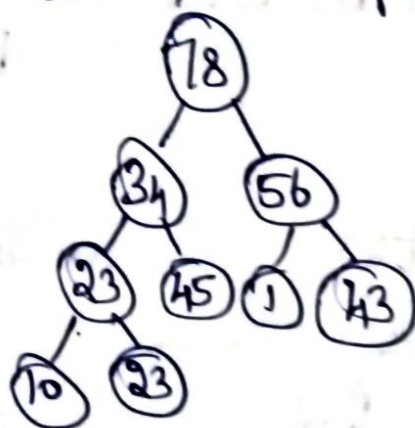
12. Insert 23



13. Insert 45



Final Max Heap



Array representation : [78, 34, 56, 23, 45, 1, 43, 10, 23]

The final heap → [78, 56, 45, 34, 23, 12, 23, 12, 23, 10, 15, 21, 6, 43]

b) B Tree of Order 3.  
 Elements: 34, 26, 45, 12, 87, 56, 78, 22, 23, 24, 65, 85, 1

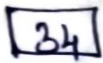
For a B-Tree of Order 3:

\* Maximum children = 3

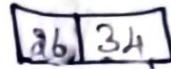
\* Maximum keys per node = 2

\* When a node gets 3 keys, the middle key is promoted.

1. Insert 34



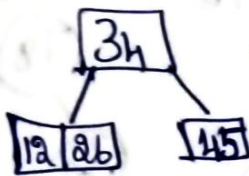
2. Insert 26



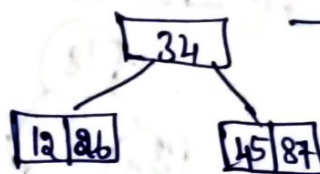
3. Insert 45 → Split



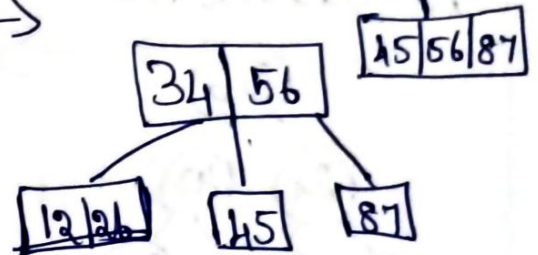
4. Insert 12



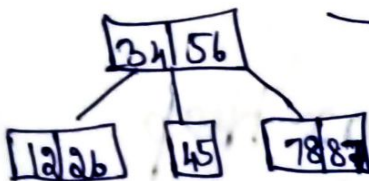
5. Insert 87



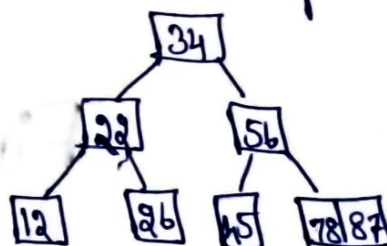
6. Insert 56 → Split



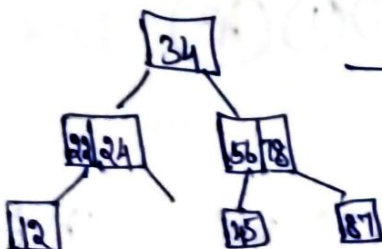
7. Insert 78



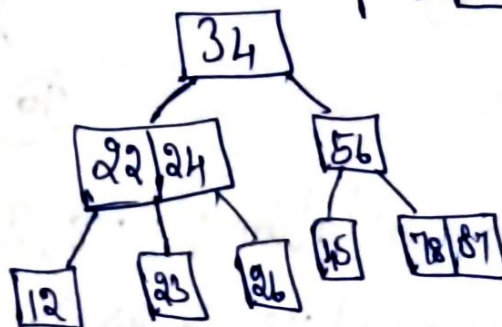
8. Insert 22 → Splits occur



9. Insert 23



10. Insert 24 → Split

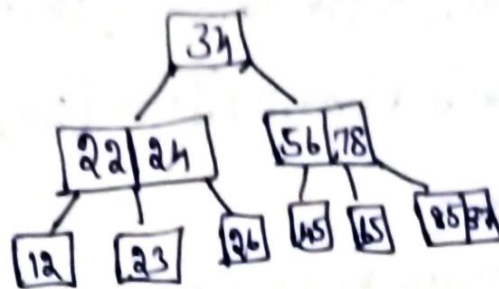




Insert 65

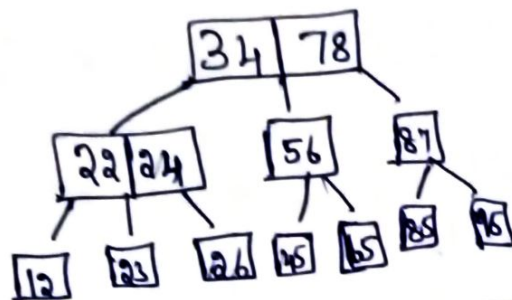


12. Insert 85



13. Insert 95 → Split and promote

Final B-Tree



7) Gm: 40, 20, 70, 14, 60, 61, 97, 30

Step 1: Divide

[40, 20, 70, 14, 60, 61, 97, 30]

↓

[40, 20, 70, 14] [60, 61, 97, 30]

↓

[40, 20] [70, 14]

↓

[60, 61] [97, 30]

↓

[40] [20]

↓

[70] [14]

↓

[60] [61]

↓

[97] [30]

Step 2: Merge

Iteration 1:

[40] + [20] → [20, 40]

[60] + [61] → [60, 61]

[70] + [14] → [14, 70]

[97] + [30] → [30, 97]

Iteration 2:

$$[20, 40] + [14, 70] \rightarrow [14, 20, 40, 70]$$

$$[60, 61] + [30, 97] \rightarrow [30, 60, 61, 97]$$

Iteration 3:

$$[14, 20, 40, 70] + [30, 60, 61, 97] \rightarrow [14, 20, 30, 40, 60, 61, 70, 97]$$

Final Sorted List:

14, 20, 30, 40, 60, 61, 70, 97

8) Quick Sort

Gm:

78, 23, 31, 88, 43, 55, 67, 55

Step 1: Pivot = 55

Elements smaller than 55  $\rightarrow$  23, 31, 43

Elements greater than 55  $\rightarrow$  78, 88, 67

|    |    |    |
|----|----|----|
| 23 | 31 | 43 |
|----|----|----|

 55 

|    |    |    |    |
|----|----|----|----|
| 78 | 88 | 67 | 55 |
|----|----|----|----|

Step 2: Sort Left Part [23, 31, 43]

Pivot = 43

|    |    |
|----|----|
| 23 | 31 |
|----|----|

 43

Pivot = 31

|    |
|----|
| 23 |
|----|

 31

So: 

|    |    |    |
|----|----|----|
| 23 | 31 | 43 |
|----|----|----|

Step 3: Sort Right Part 

|    |    |    |    |
|----|----|----|----|
| 78 | 88 | 67 | 55 |
|----|----|----|----|

Pivot = 55

55 

|    |    |    |
|----|----|----|
| 78 | 88 | 67 |
|----|----|----|



sort [78|88|67]

pivot = 67

67 [78|88]

Sort [78|88]

Pivot = 88

[78]

~~Pivot = 88~~ So: [55|67|78|88]

Final Sorted List  $\rightarrow 23, 31, 43, 55, 55, 67, 78, 88$

9) Assuming the hash function is:  
 $h(K) = K \bmod 10.$

Given keys:

0, 1, 81, 4, 64, 25, 16, 36, 9, 49.

1. Open Hashing (Separate chaining)

| Key | Hash value |
|-----|------------|
| 0   | 0          |
| 1   | 1          |
| 81  | 1          |
| 4   | 4          |
| 64  | 4          |
| 25  | 5          |
| 16  | 6          |
| 36  | 6          |
| 9   | 9          |
| 49  | 9          |

## 2. Linear Probing

$$h(K) = K \bmod 10.$$

| Key | Hash | Position        |
|-----|------|-----------------|
| 0   | 0    | 0               |
| 1   | 1    | 1               |
| 81  | 1    | 2               |
| 4   | 4    | 4               |
| 64  | 4    | 5               |
| 25  | 5    | 6               |
| 16  | 6    | 7               |
| 36  | 6    | 8               |
| 9   | 9    | 9               |
| 49  | 9    | 3 (wrap around) |

## 10) BFS Traversal

From the given graph, the connections are:

\*  $0 \rightarrow 1, 3, 2$

\*  $3 \rightarrow 4$

\*  $2 \rightarrow 5$

Starting BFS from node 0 and visiting left to right:

Step-by-Step

(1) Start at 0 (2) Visit neighbours of  $0 \rightarrow 1, 3, 2$

(3) Visit neighbour of  $3 \rightarrow 4$

(4) Visit neighbour of  $2 \rightarrow 5$

BFS Traversal  $\rightarrow 0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 4 \rightarrow 5$